Unit-5

# Exception Handling in Java

## ❖ Introduction

During the execution of a Java program, unexpected situations may occur. Some problems are so serious that the program cannot recover from them, while others can be handled by the programmer. Java classifies these problems into Errors and Exceptions.

## ❖ What is an Error?

An Error is a serious problem that occurs during program execution and is generally beyond the control of the programmer.
Errors usually occur because of:

- Lack of system resources
- JVM failure
- Hardware failure
- Memory overflow

Errors are not meant to be handled by application programs.

**Examples**

- OutOfMemoryError
- StackOverflowError
- VirtualMachineError
- AssertionError

## ❖ What is an Exception?

An Exception is an abnormal condition that occurs during program execution and can be handled by the programmer.
Examples:

- Division by zero
- Invalid array index
- File not found
- Invalid user input

**Example**

```
int a = 10;
int b = 0;
System.out.println(a / b);
```

**Output:** ArithmeticException

❖ **Error vs Exception**

| Error | Exception |
|---|---|
| Serious problem | Recoverable problem |
| Cannot usually be handled | Can be handled |
| Occurs due to JVM/System | Occurs due to programming mistakes |
| Package: java.lang.Error | Package: java.lang.Exception |
| Example: OutOfMemoryError | Example: ArithmeticException |

❖ **Example Without Exception Handling**

```
public class Demo {
    public static void main(String[] args) {
        int a = 10;
        int b = 0;
        int result = a / b;
        System.out.println(result);
        System.out.println("Program Ended");
    }
}
```

**Output:** Exception in thread "main" java.lang.ArithmeticException: / by zero

❖ **Types of Exceptions in Java**

In Java, exceptions are broadly classified into two types:

1. Checked Exceptions (Compile-Time Exceptions)
2. Unchecked Exceptions (Runtime Exceptions)

**1. Checked Exceptions (Compile-Time Exceptions)**

A Checked Exception is an exception that is checked by the compiler during compilation. The programmer must handle these exceptions using either a try-catch block or the throws keyword. Otherwise, the program will not compile.

**Characteristics**

- Checked at compile time.
- Mandatory to handle.
- Inherits directly from the Exception class (excluding RuntimeException).

❖ **Common Checked Exceptions**

| Exception | Description |
|---|---|
| IOException | Error while performing input/output operations. |
| FileNotFoundException | File does not exist. |
| SQLException | Error while accessing a database. |
| ClassNotFoundException | Specified class cannot be found. |
| InterruptedException | A thread is interrupted during execution. |

**Example**

```
import java.io.FileReader;
import java.io.IOException;
public class CheckedExceptionDemo {
	public static void main(String[] args) {
		try {
			FileReader file = new FileReader("student.txt");
			System.out.println("File Opened Successfully");
		}
		catch (IOException e) {
			System.out.println("File not found.");
		 }
	}
}
```

**2. Unchecked Exceptions (Runtime Exceptions)**

An Unchecked Exception occurs during program execution (runtime). The compiler does not check these exceptions, and handling them is optional.

**Characteristics**

- Occurs at runtime.
- Not checked by the compiler.
- Usually caused by programming mistakes.
- Inherits from the RuntimeException class.

❖ **Common Unchecked Exceptions**

| **Exception** | **Description** |
|---|---|
| ArithmeticException | Division by zero. |
| NullPointerException | Accessing a null object. |
| ArrayIndexOutOfBoundsException | Invalid array index. |
| NumberFormatException | Invalid number conversion. |
| StringIndexOutOfBoundsException | Invalid string index. |
| ClassCastException | Invalid object casting. |

**Example**

```
public class UncheckedExceptionDemo {
        public static void main(String[] args) {
                int a = 20;
                int b = 0;
                System.out.println(a / b);
        }
}
```

**Output:** Exception in thread "main" java.lang.ArithmeticException: / by zero

### ❖ try-catch Block

The try-catch block is used to handle exceptions in Java.

- try block contains the code that may generate an exception.
- catch block contains the code that handles the exception.

**Syntax:**

```
try {
    // Risky code
}
catch(ExceptionType e) {
  // Exception handling code
}
```

**Example**

```
public class TryCatchDemo {
        public static void main(String[] args) {
                try {
                        int arr[] = {10,20,30};
                        System.out.println(arr[5]);
                }
                catch(ArrayIndexOutOfBoundsException e) {
                        System.out.println("Invalid Array Index");
                }
                        System.out.println("Program Continues");
        }
}
```

**Output:** Invalid Array Index
Program Continues

**Advantages**

- Prevents abnormal program termination.
- Makes the program more reliable.
- Helps in debugging.
- Allows graceful error recovery.

❖ **Multiple catch Blocks**

- A single try block may generate different types of exceptions.
- Java allows multiple catch blocks to handle different exceptions separately.

**Syntax**

```
try{

}
catch(Exception1 e){

}
catch(Exception2 e){

}
catch(Exception3 e){

}
```

**Example**

```
public class MultipleCatchDemo {
    public static void main(String[] args) {
        try {
            int arr[] = new int[3];
            arr[5] = 100;
            int x = 20 / 0;
        }
        catch(ArrayIndexOutOfBoundsException e) {
            System.out.println("Array Index Error");
        }
        catch(ArithmeticException e) {
            System.out.println("Arithmetic Error");
        }
    }
}
```

**Output:** Array Index Error

❖ **finally Block**

- The finally block always executes whether an exception occurs or not.
- It is mainly used to release system resources.

**Uses**

- Closing files
- Closing database connections
- Closing scanners
- Closing sockets

**Syntax**

```
try{

}
catch(Exception e){

}
finally{

}
```

**Example**

```
public class FinallyDemo {
        public static void main(String[] args) {
                try {
                        int x = 20 / 0;
                }
                catch(Exception e) {
                System.out.println("Exception Handled");
                }
                finally {
                System.out.println("Finally Block Executed");
                }
        }
}
```

**Output:** Exception Handled
Finally Block Executed

### ❖ throw Keyword

The throw keyword is used to manually create and throw an exception.

**Syntax**

```
throw new ExceptionName("Message");
```

**Example**

```
public class ThrowDemo {
    public static void main(String[] args) {
        int age = 16;
        if(age < 18) {
            throw new ArithmeticException("Not Eligible for Voting");
        }
        else {
            System.out.println("Eligible");
        }
    }
}
```

**Output:**

```
Exception in thread "main"
java.lang.ArithmeticException:
Not Eligible for Voting
```

**Applications**

- Input validation
- Business rules
- Age validation
- Password validation

### ❖ throws Keyword

The throws keyword is used in the method declaration to indicate that the method may throw one or more exceptions.

**Syntax**

```
returnType methodName()
throws ExceptionName
```

**Example**

```
import java.io.*;
public class ThrowsDemo {
        static void readFile()
        throws IOException {
                FileReader file = new FileReader("student.txt");
        }
public static void main(String[] args)
        throws IOException {
                readFile();
        }
}
```

❖ **Why Use throws?**

Instead of handling an exception immediately, we can pass the responsibility to the calling method.

❖ **throw vs throws**

| throw | throws |
|---|---|
| Used inside a method | Used in method declaration |
| Throws one exception object | Declares one or more exception classes |
| Followed by an object | Followed by class names |
| Used to create an exception manually | Used to pass responsibility to caller |

❖ **Custom Exceptions**

A Custom Exception is a user-defined exception created by extending the Exception class.

❖ **Why Create Custom Exceptions?**

Java's built-in exceptions are not enough for business logic.

**Examples**

- InvalidAgeException
- InsufficientBalanceException
- InvalidPasswordException

❖ **Steps**

**Step 1:** Create Exception Class

```
        class InvalidAgeException extends Exception {
         InvalidAgeException(String message) {
                super(message);
        }
}
```

**Step 2:** Throw Exception

```
public class CustomExceptionDemo {
static void checkAge(int age)
        throws InvalidAgeException {
                if(age < 18)
                        throw new InvalidAgeException("Age must be 18 or above.");
                else
                        System.out.println("Eligible");
        }

        public static void main(String[] args) {
                try {
                        checkAge(15);
                }
                catch(InvalidAgeException e) {
                        System.out.println(e.getMessage());
                }
        }

}
```

**Output:** Age must be 18 or above.

❖ **Best Practices of Exception Handling**

**1. Catch Specific Exceptions**

Catch the exact exception instead of the generic Exception.

```
try {
        int result = 10 / 0;
}
catch (ArithmeticException e) {
        System.out.println("Cannot divide by zero.");
}
```

**2. Do Not Leave Catch Block Empty**

Always handle or print the exception.

```
try {
        int result = 10 / 0;
}
catch (ArithmeticException e) {
        System.out.println(e.getMessage());
}
```

**3. Use Meaningful Error Messages**

Display clear messages instead of generic ones.

```
System.out.println("Invalid age. Age must be 18 or above.");
```

**4. Keep the try Block Small**

Only place risky code inside the try block.

```
try {
    int result = a / b;
}
catch (ArithmeticException e) {
    System.out.println("Division by zero is not allowed.");
}
```

**5. Use finally for Resource Cleanup**

Always close resources in the finally block.

```
Scanner sc = new Scanner(System.in);
try {
    System.out.println(sc.nextInt());
}
finally {
    sc.close();
}
```

**6. Use throw for Business Validation**

Throw an exception when a business rule is violated.

```
int age = 16;
if (age < 18) {
    throw new IllegalArgumentException("Age must be 18 or above.");
}
```

**7. Use throws to Delegate Exception Handling**

Declare exceptions in the method signature.

```
import java.io.IOException;
public class Demo {

    static void readFile() throws IOException {
    // File reading code
    }
}
```

**8. Create Custom Exceptions**

Create user-defined exceptions for application-specific errors.

```
class InvalidAgeException extends Exception {
        InvalidAgeException(String msg) {
                super(msg);
        }
}
```

**9. Do Not Use Exceptions for Normal Program Flow**

Use conditions instead of exceptions whenever possible.

```
if (number != 0) {
        System.out.println(10 / number);
} else {
        System.out.println("Cannot divide by zero.");
}
```

**10. Log Exceptions**

Print or log exception details for debugging.

```
catch (Exception e) {
        e.printStackTrace();
}
```

**11. Catch Specific Exceptions Before General Exceptions**

```
try {
        int result = 10 / 0;
}
catch (ArithmeticException e) {
        System.out.println("Arithmetic Error");
}
catch (Exception e) {
        System.out.println("General Error");
}
```

**12. Never Ignore Exceptions**

Do not leave the catch block empty.

```
catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
}
```

## Important Questions

1. Explain Exception Handling in Java with a suitable example.
2. Explain the Exception Hierarchy in Java. Differentiate between Errors and Exceptions.
3. Explain the try-catch-finally mechanism with syntax, flow diagram, and example.
4. Explain the throw and throws keywords with suitable examples. Differentiate between them.
5. Explain Multiple Catch Blocks with syntax, rules, and example.
6. Explain the finally block. Why is it used? Write a Java program demonstrating its use.
7. Write a Java program to demonstrate exception handling using multiple exceptions.
8. Explain the Best Practices of Exception Handling in Java with examples.
9. Explain the complete Exception Handling mechanism in Java with suitable examples.
10. Explain different types of exceptions in Java with examples.
11. Explain the advantages and applications of Exception Handling in Java.
12. Write a Java program to create a user-defined exception for age validation.
13. Explain all Exception Handling keywords (try, catch, finally, throw, throws) with syntax and examples.
14. Explain Custom Exceptions in Java. How do you create and use a custom exception? Write a Java program.
15. Explain Checked Exceptions and Unchecked Exceptions with suitable examples. Compare both.

*****